Structured output (a checked answer, not a hope) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `structured_output_practice.py`: asking for "JSON, please" in a prompt is only a request. A Pydantic model sent with `.parse()` is a form with locked fields: the model can only fill in the blanks you defined, in the types you defined. This exercise practices that form — first by hand, then with the model filling it. **If not:** the Build Task's `schemas.py`, Project 1's triage agent and Doc05's `.with_structured_output()` would be the first time you ever met a Pydantic model.

All examples below need `pip install openai python-dotenv pydantic` and your key in `.env`.

Basic Version

Approach 1 — one form, one hand check, one call

```
# structured_output_practice.py
from typing import Literal

from dotenv import load_dotenv
from openai import OpenAI
from pydantic import BaseModel, ValidationError

load_dotenv()
client = OpenAI()


class ExtractedInfo(BaseModel):
    name: str | None
    intent: str | None
    urgency: Literal["low", "normal", "urgent"]


# part 1: Pydantic checks a dict by hand — no API call
good = {"name": "Maria", "intent": "reset her password", "urgency": "urgent"}
print(ExtractedInfo.model_validate(good))
```

```
bad = {"name": "Maria", "intent": "reset her password", "urgency": "very"}
try:
    ExtractedInfo.model_validate(bad)
except ValidationError:
    print("bad urgency -> ValidationError")

# part 2: the model fills the same form
completion = client.beta.chat.completions.parse(
    model="gpt-4o-mini",
    messages=[
        {"role": "user", "content": "I'm Maria, reset my password today!"},
    ],
    response_format=ExtractedInfo,
)
message = completion.choices[0].message
if message.refusal:
    print(f"The model refused: {message.refusal}")
else:
    info = message.parsed
    print(info)
    print(f"name: {info.name}, urgency: {info.urgency}")
```

Expected output (model values vary):

```
name='Maria' intent='reset her password' urgency='urgent'
bad urgency -> ValidationError
name='Maria' intent='reset her password' urgency='urgent'
name: Maria, urgency: urgent
```

This works for one clear message. But nothing is reusable, there's no system prompt telling the model not to guess, and a message like `"the sky is blue"` would still come back as an object full of `None` that the caller has to inspect.

Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

Approach 1 — `extract_info()` returns a checked object or `None`

Story — `extract_info()`: the caller should never have to know about refusals, validation errors or empty answers. One function turns all three into `None`, and everything else into an `ExtractedInfo` it can trust. **If not:** every caller would repeat the same three checks — and the one that forgets the refusal check reads `parsed` as `None` and crashes.

```
# structured_output_practice.py
from typing import Literal

from dotenv import load_dotenv
from openai import OpenAI
from pydantic import BaseModel, ValidationError

load_dotenv()
client = OpenAI()

SYSTEM_PROMPT = (
    "Pull the person's name, what they want, and how urgent it is out "
    "of the message. If there is no name or no request, use null — "
    "never guess."
)


class ExtractedInfo(BaseModel):
    # why: str | None, not str — a required name would force the model
    # to invent one when the message has none
    name: str | None
    # why: a message can ask for nothing at all ("the sky is blue")
    intent: str | None
    # why: Literal — the model can only pick one of these 3 exact values
    urgency: Literal["low", "normal", "urgent"]


def check_by_hand() -> None:
    # how: model_validate() is the same check .parse() runs for you on
    # the model's JSON — here you run it yourself on a plain dict
    good = {"name": "Maria", "intent": "reset password", "urgency": "urgent"}
    info = ExtractedInfo.model_validate(good)
```

```
    print(f"case 1 (good dict)    -> {info.urgency}")

    bad = {"name": "Maria", "intent": "reset password", "urgency": "very"}
    try:
        ExtractedInfo.model_validate(bad)
        print("case 2 (bad urgency)  -> accepted (wrong!)")
    except ValidationError:
        print("case 2 (bad urgency)  -> ValidationError")


def extract_info(text: str) -> ExtractedInfo | None:
    try:
        completion = client.beta.chat.completions.parse(
            model="gpt-4o-mini",
            messages=[
                {"role": "system", "content": SYSTEM_PROMPT},
                {"role": "user", "content": text},
            ],
            response_format=ExtractedInfo,
        )
    except ValidationError:
        # when: the reply didn't fit the form — rare with .parse(), but
        # the caller should get "nothing", not a crash
        return None

    message = completion.choices[0].message
    # why: check refusal before parsed — when the model declines,
    # parsed is None and refusal holds the reason
    if message.refusal:
        print(f"The model refused: {message.refusal}")
        return None

    info = message.parsed
    # when: no name and no request — report "nothing found" instead of
    # handing back an object full of None
    if info.name is None and info.intent is None:
        return None
    return info


def main() -> None:
    check_by_hand()
    texts = [
        "Hi, I'm Maria and I need my password reset today, please!",
        "Can someone check why my invoice is wrong?",
        "the sky is blue",
    ]
```

```
    for text in texts:
        info = extract_info(text)
        print()
        print(f"Text: {text}")
        if info is None:
            print("  nothing to extract")
        else:
            print(f"  name:    {info.name}")
            print(f"  intent:  {info.intent}")
            print(f"  urgency: {info.urgency}")


if __name__ == "__main__":
    main()
```

Expected output (model values vary):

```
case 1 (good dict)    -> urgent
case 2 (bad urgency)  -> ValidationError

Text: Hi, I'm Maria and I need my password reset today, please!
  name:    Maria
  intent:  reset her password
  urgency: urgent

Text: Can someone check why my invoice is wrong?
  name:    None
  intent:  check a wrong invoice
  urgency: normal

Text: the sky is blue
  nothing to extract
```

The second text has no name, so `name` is `None` — the model reported "no name" instead of inventing one. That's what `str | None` plus the system prompt buys you.

Difference from Basic: a system prompt that forbids guessing; comments on why each field has its type; and `extract_info()`, which gives the caller exactly two cases — a trusted object, or `None` for a refusal, a bad reply or an empty answer.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Approach 2 — an `/extract` command in a small loop

Story — the command loop: a real app doesn't hardcode its texts; the user types them. A line that starts with `/extract` is a request for this feature, and the rest of the line is the text. `.startswith()` checks how a string begins, and `.removeprefix()` cuts that beginning off. **If not:** the Build Task's `/extract` command would be the first time you ever split a command from its text.

```
# structured_output_practice.py
from typing import Literal

from dotenv import load_dotenv
from openai import OpenAI
from pydantic import BaseModel, ValidationError

load_dotenv()
client = OpenAI()

SYSTEM_PROMPT = (
    "Pull the person's name, what they want, and how urgent it is out "
    "of the message. If there is no name or no request, use null — "
    "never guess."
)


class ExtractedInfo(BaseModel):
    # why: str | None, not str — a required name would force the model
    # to invent one when the message has none
    name: str | None
    # why: a message can ask for nothing at all ("the sky is blue")
    intent: str | None
    # why: Literal — the model can only pick one of these 3 exact values
    urgency: Literal["low", "normal", "urgent"]


def extract_info(text: str) -> ExtractedInfo | None:
    try:
        completion = client.beta.chat.completions.parse(
            model="gpt-4o-mini",
            messages=[
                {"role": "system", "content": SYSTEM_PROMPT},
                {"role": "user", "content": text},
```

```
            ],
            response_format=ExtractedInfo,
        )
    except ValidationError:
        # when: the reply didn't fit the form — rare with .parse(), but
        # the caller should get "nothing", not a crash
        return None

    message = completion.choices[0].message
    # why: check refusal before parsed — when the model declines,
    # parsed is None and refusal holds the reason
    if message.refusal:
        print(f"The model refused: {message.refusal}")
        return None

    info = message.parsed
    # when: no name and no request — report "nothing found" instead of
    # handing back an object full of None
    if info.name is None and info.intent is None:
        return None
    return info


def main() -> None:
    # how: a small command loop — the same shape the Build Task's
    # main.py uses for its /extract feature
    while True:
        user_input = input("You: ")
        if user_input == "/quit":
            break

        # how: .startswith() checks how the text begins, and
        # .removeprefix() returns the text with that beginning cut off
        if user_input.startswith("/extract "):
            text = user_input.removeprefix("/extract ")
            info = extract_info(text)
            if info is None:
                print("  nothing to extract")
            else:
                print(f"  name: {info.name}, intent: {info.intent}")
                print(f"  urgency: {info.urgency}")
        else:
            print("  type /extract <text>, or /quit")


if __name__ == "__main__":
    main()
```

Expected output (you type the lines after `You:`; model values vary):

```
You: /extract Hi, I am Maria and I need my password reset today!
  name: Maria, intent: reset her password
  urgency: urgent
You: /extract the sky is blue
  nothing to extract
You: hello
  type /extract <text>, or /quit
You: /quit
```

Difference from Approach 1: the texts come from the user instead of a fixed list, and `.startswith()` / `.removeprefix()` turn one typed line into a command plus its text — exactly how the Build Task's `main.py` handles `/extract`.

Which one should you actually write? Intermediate's `extract_info()` is the shape this document's Build Task uses as `get_structured_reply()` — and Approach 2's command loop is its `/extract` command — and Project 1's triage agent uses the same checks. Remember the shape is guaranteed but the **truth** is not: the model can still pick the wrong urgency — check values that matter.